

Software Engineering: Discipline, Process, and Ethics

Managing complexity and risk in
modern system development



Based on Software Engineering (9th Edition) by Ian Sommerville

Engineering is about practical results, not just theory.

Computer Science

Focuses on theory and fundamentals.

System Engineering

Concerned with all aspects of computer-based systems (hardware, software, policy, process).

Software Engineering

The engineering discipline concerned with all aspects of software production, from specification to maintenance.

Maintainability



Software must evolve to meet changing needs.

Dependability



Reliability, safety, and security.
No damage upon failure.

Essential Attributes

Efficiency



No wasteful use of resources.

Acceptability



Understandable, usable,
and compatible.

The code of ethics is as rigid as the code in the compiler.

Software engineers have significant opportunities to do good or cause harm. — ACM/IEEE Joint Task Force



Public interest
Client interest
High standards
Independence
Ethical management
Profession integrity
Supportive colleagues
Lifelong learning

Three distinct systems; three distinct engineering challenges.



Insulin Pump Control

Type: Embedded System.
Context: **Safety-critical.**
Failure results in health damage or death.



MHC-PMS (Mental Health)

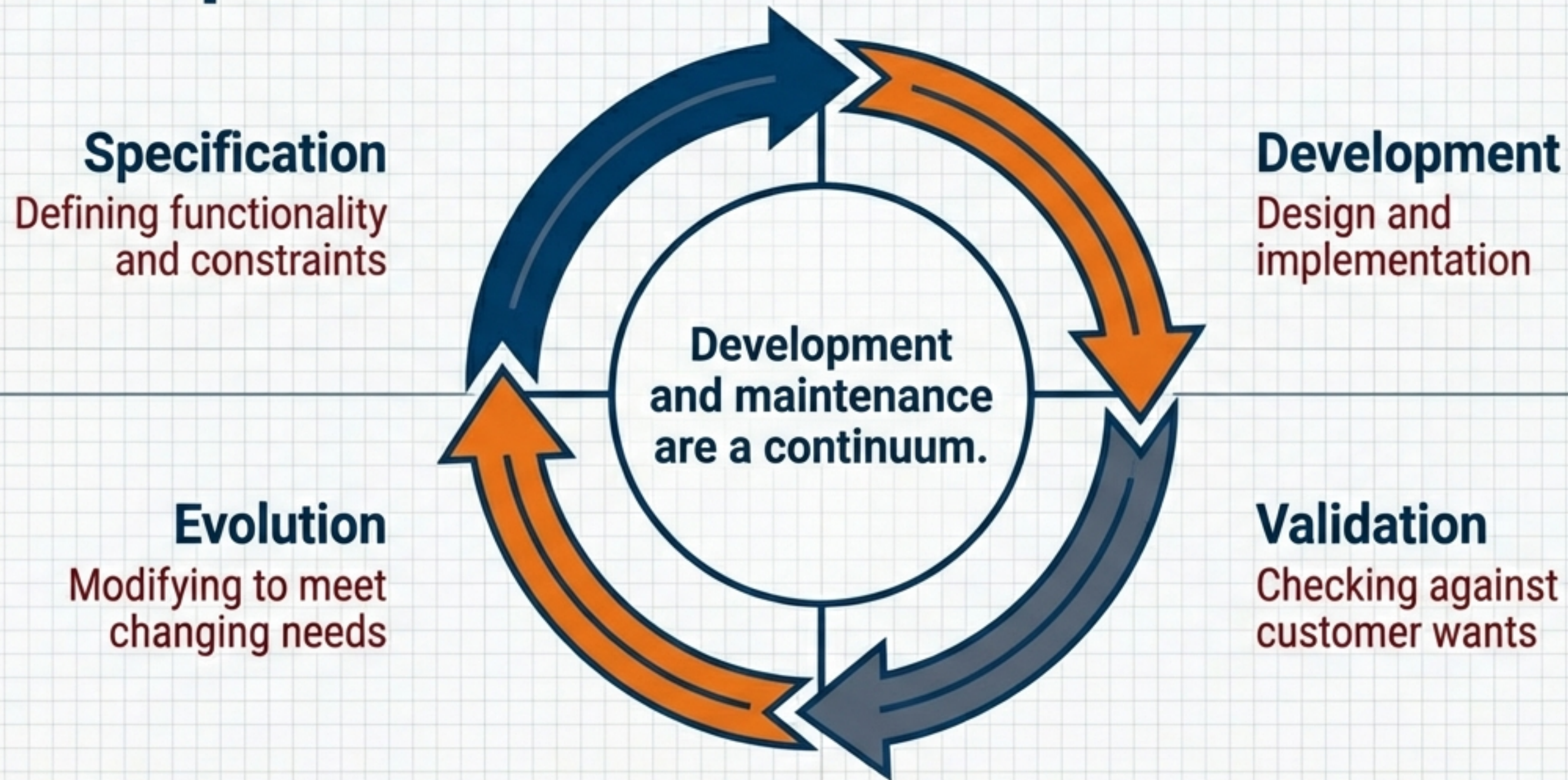
Type: Information System.
Context: **Privacy-critical.**
Management of patient records across clinics.



Wilderness Weather Station

Type: Sensor-based Collection.
Context: **Maintenance-critical.**
Remote, hostile environment, autonomous power.

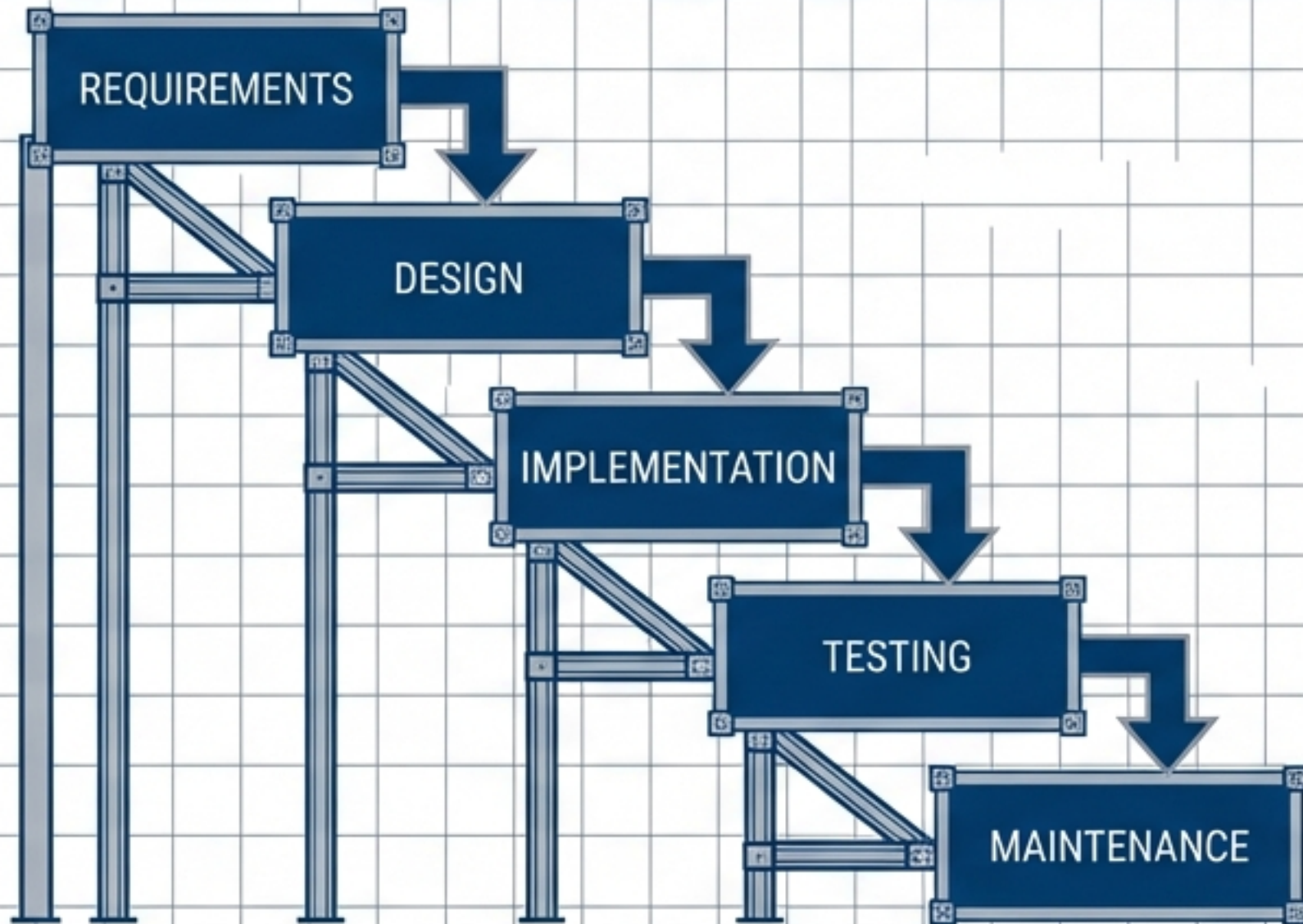
The four fundamental activities of the software process.



Regardless of the method (Agile or Waterfall), these four steps must happen.

Structural Models: Plan-Driven vs. Incremental.

THE WATERFALL MODEL



The Waterfall Model: Best for well-understood requirements. **Rigid.** Risk of rework.

INCREMENTAL DEVELOPMENT

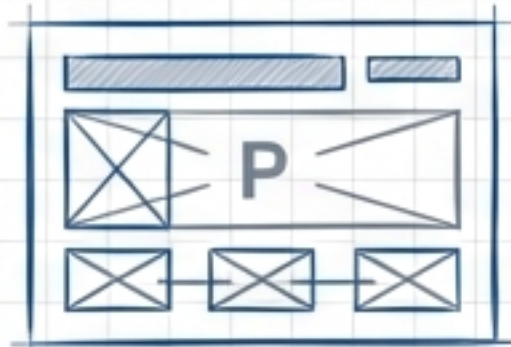


Incremental Development: Interleaved activities. Flexible. **Reduced cost of change.**

Reducing the cost of Rework.

Change is inevitable. Strategies to mitigate risk:

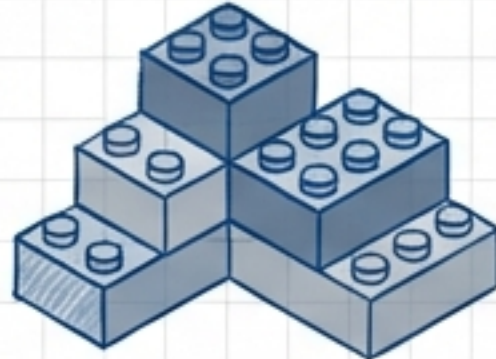
Change Avoidance



Change Avoidance

Prototyping: Validate requirements early with throw-away versions.

Change Tolerance



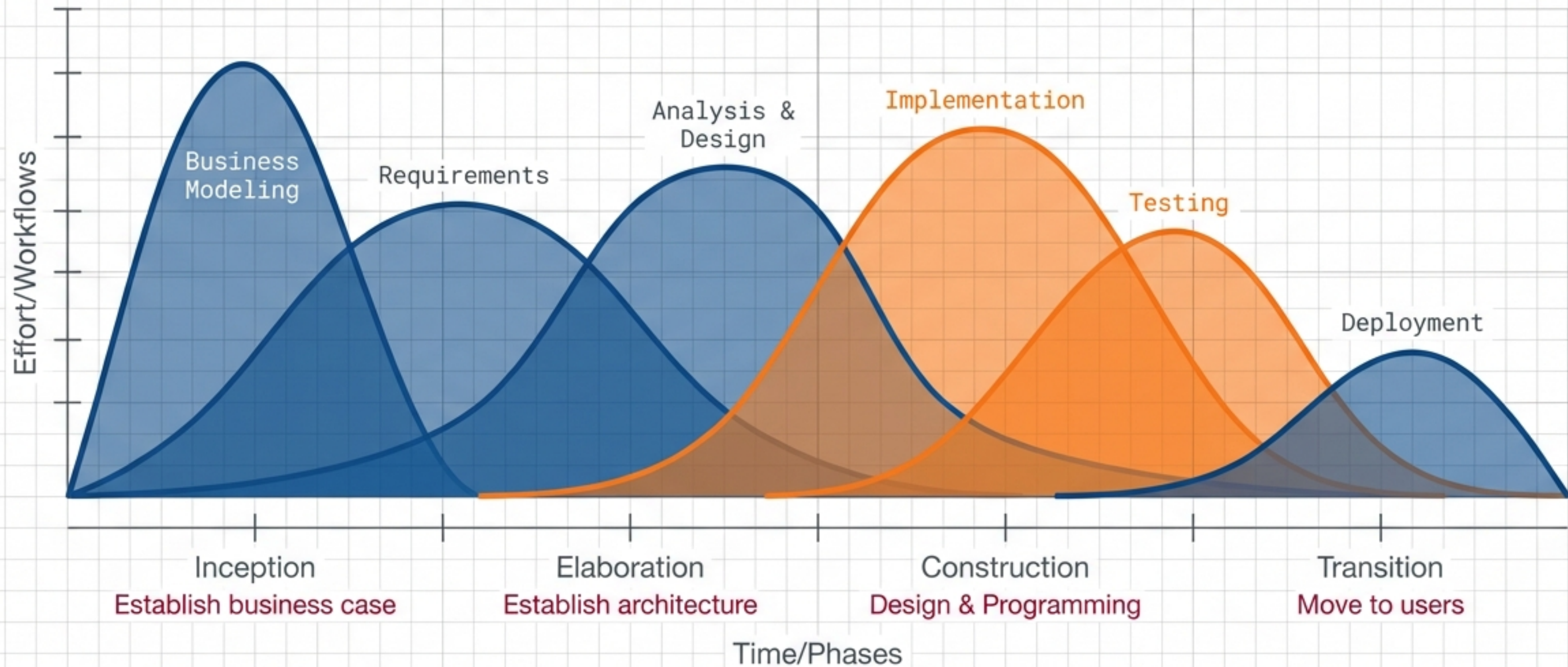
Change Tolerance

Incremental Delivery: Deploy increments for user experimentation.

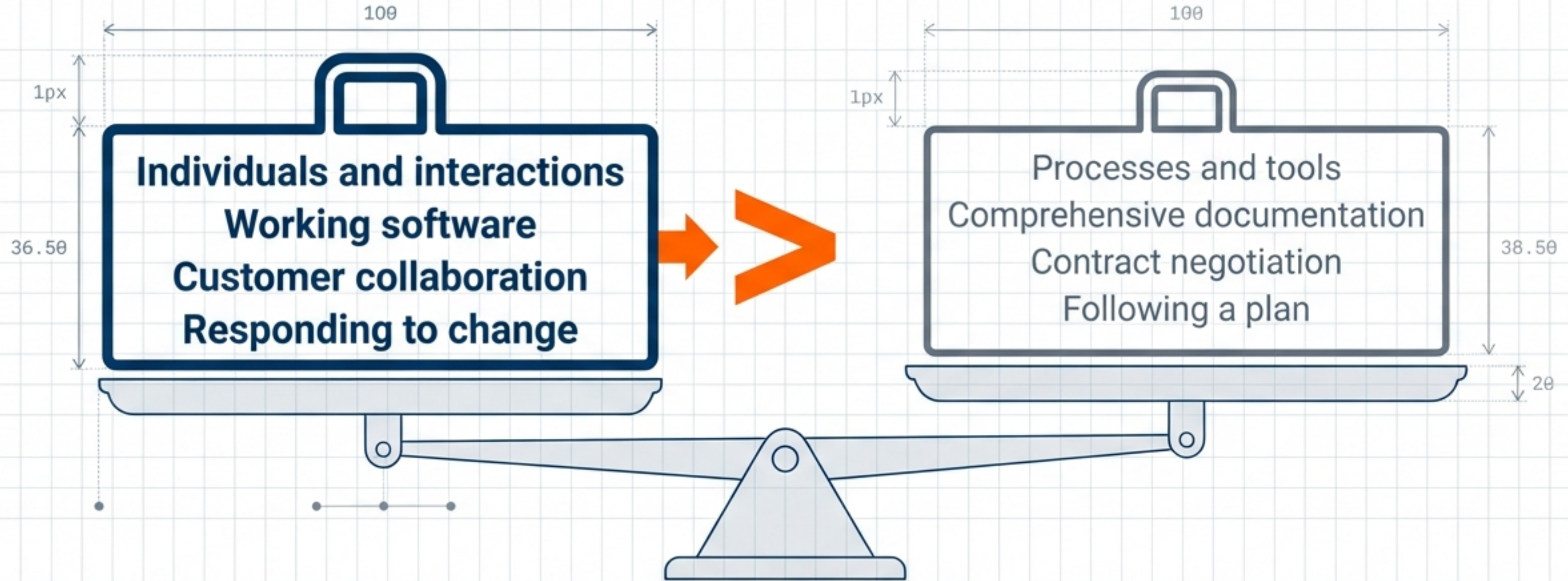


Boehm's Spiral Model: A risk-driven framework combining avoidance and tolerance.

The Rational Unified Process (RUP): A hybrid standard.

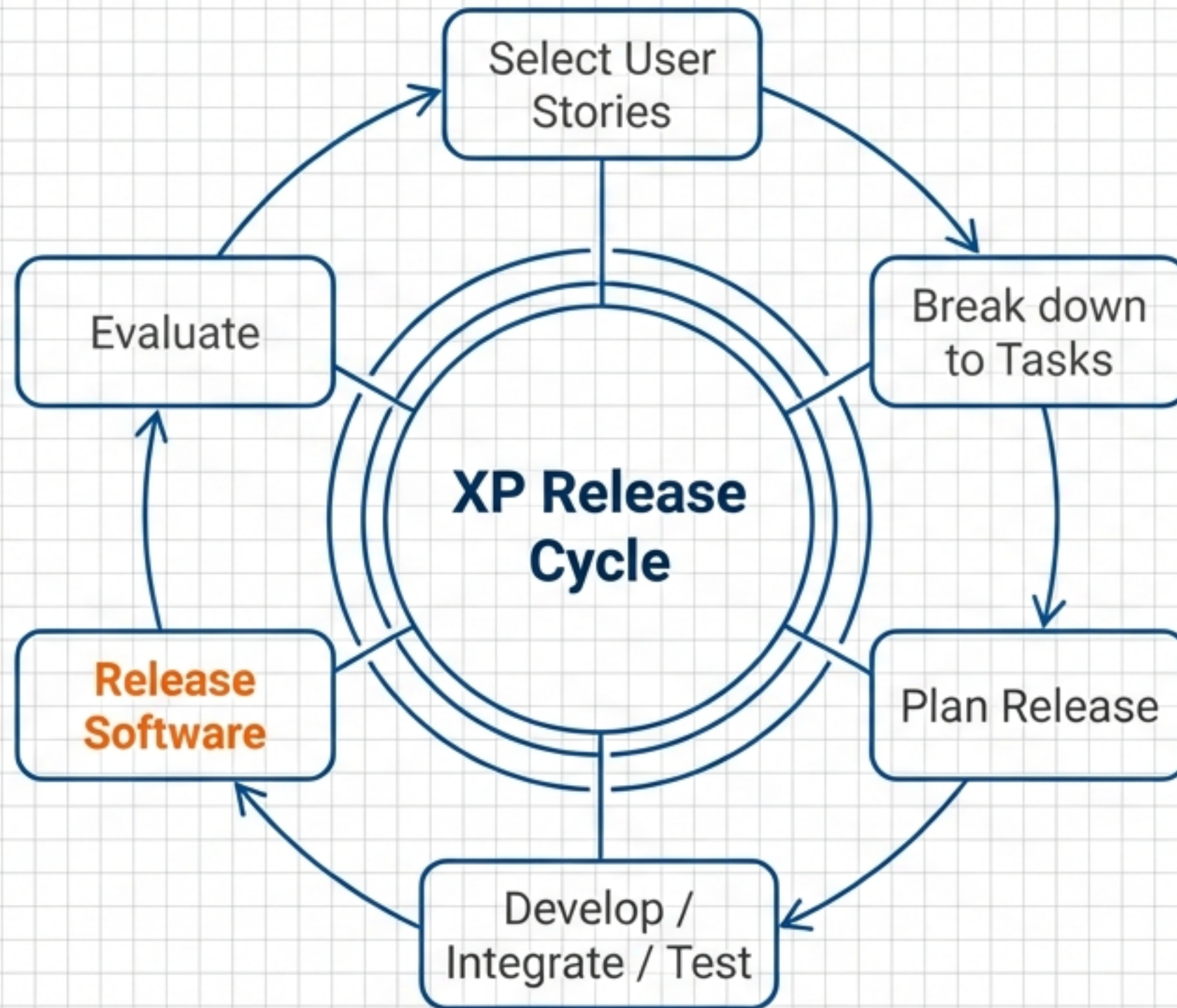


The Agile Manifesto: A shift in value.



Agile methods (XP, Scrum) rely on **incremental development** and are best for **rapidly changing business environments**.

Extreme Programming (XP): Pushing good practice to the limit.



Key Practices



Test-First Development:
Tests written before code.



Pair Programming:
Collective ownership.

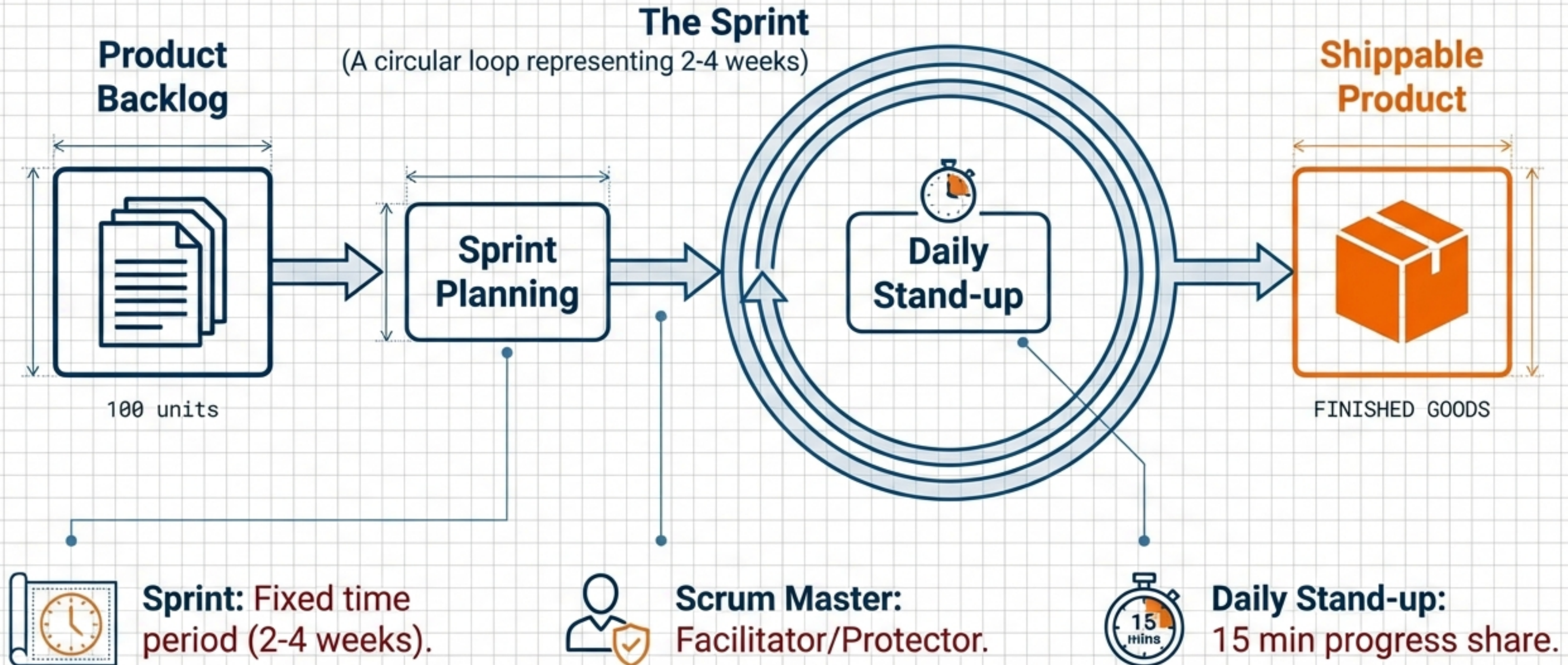


Refactoring:
Constant code improvement.



Small Releases:
Frequent delivery.

Scrum: Managing the chaos of iteration.



The Hybrid Reality: Scaling Agile.

Agile is not a silver bullet. Large “Brownfield” systems often require hybrid approaches.

Factor	Plan-Driven	Agile
Team Size	Large, Distributed	Small, Co-located
Criticality	Safety-Critical (e.g. Insulin Pump)	Low Risk (e.g. Web App)
Requirements	Stable	Volatile / Changing
System Life	Long Lifetime	Short Lifetime

Scaling Up: Adding structure to Agile. Scaling Out: Cultural change in organizations.

Requirements Engineering: Defining the 'What' before the 'How'.

Translation



User Requirements

High-level abstract statements for customers/managers.

Example:
The MHC-PMS shall generate monthly management reports.

Refinement



System Requirements

Detailed description of functions for developers.

Example:
The system shall implement a **grid search** – on the patient_ID database field...

Functional Services vs. Non-Functional Constraints

Functional Requirements

Statements of services the system should provide.



Example:

MHC-PMS: A user shall be able to search appointment lists.

Non-Functional Requirements

Constraints on services (timing, security, compliance).

Critical insight: Failure here can crash the entire system.

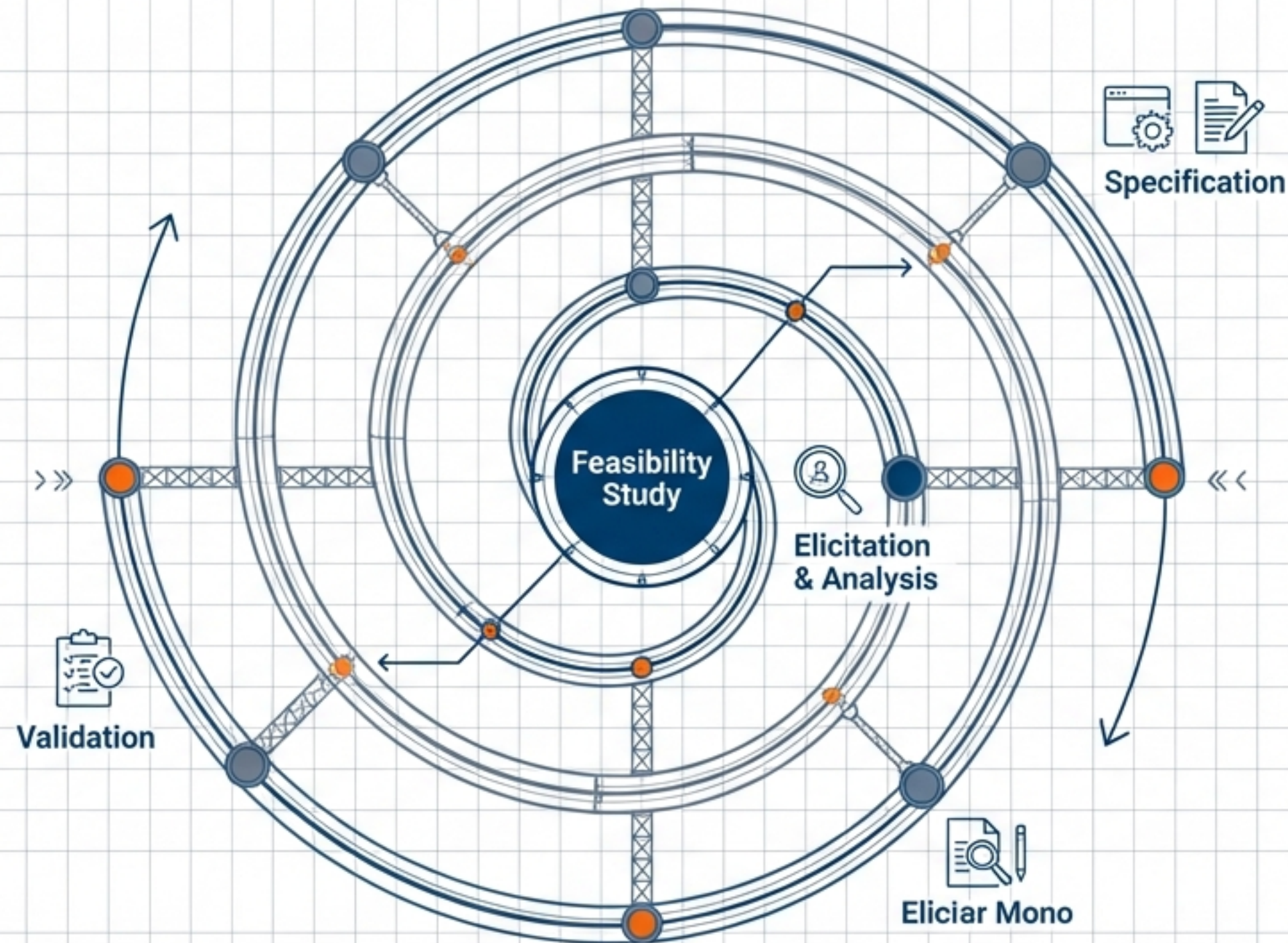


1. **Insulin Pump: Reliability** is critical. (Non-functional)



2. **MHC-PMS: Privacy** is critical. (Non-functional)

The Spiral of Requirements and Engineering.



Software Engineering is the application of a systematic, disciplined, and ethical approach.
The goal: Controlling complexity to deliver value.